Contents

01 - Getting Started 2
Prerequisites . . . 3
Build Instructions . . . 3
Project Initialization . . . 3
Your First Program . . . 3
Writing Comments . . . 3

02 - Variables and Types 4
Constant Declaration with let . . . 4
Variable Declaration with var . . . 4
Type Annotations . . . 4
Basic Types . . . 4
String Operations . . . 5
Escape Sequences . . . 5
Reassignment Rules . . . 5
Tuple Destructuring . . . 6

03 - Operators 6
Arithmetic Operators . . . 6
Comparison Operators . . . 7
Logical Operators . . . 7
Bitwise Operators . . . 8
Compound Assignment Operators . . . 8
Increment / Decrement Operators . . . 8
Type Promotion Rules . . . 9
Membership Operators . . . 9

Control Flow 10
if / elif / else . . . 10
while Loop . . . 10
for Loop and range . . . 10
break and continue . . . 11
Nesting Example . . . 12
Scope Rules . . . 12
match . . . 13

Functions 14
Basic Function Definition . . . 14
Calling Functions . . . 14
Recursive Functions . . . 14
Function Overloading . . . 15
Omitting the Return Type (Unit Type) . . . 15

Structs and Enums 15
Defining Structs with record . . . 15
Using Constructors . . . 15
Field Access (Dot Notation) . . . 16
Field Assignment . . . 16
Structs as Function Parameters . . . 16
Nested Structs . . . 16
Enums . . . 17

Collections 17

Tuples . . . 17
Lists . . . 18
Maps . . . 20
Sets . . . 21
Iterators . . . 22

Advanced Features **23**
Lambda Functions . . . 23
Closures . . . 24
Higher-Order Functions . . . 24
Functions as Values . . . 24
UFCS (Uniform Function Call Syntax) . . . 24
Operator Overloading . . . 25
Option Type . . . 25
Concurrency Basics . . . 26
F-String (String Interpolation) . . . 26
Type Casting (`as`) . . . 27
Enum with Associated Data (ADT) . . . 27
Generic Enum . . . 27
Result Type . . . 28

Packages **28**
from/import Syntax . . . 28
Subdirectories (Dot-Separated Paths) . . . 28
Directory Packages . . . 29
Standard Library (`std`) . . . 29
Search Path Priority . . . 29
RY_PATH Environment Variable . . . 29
Limitations . . . 30

Design by Contract **30**
Preconditions (`require`) . . . 30
Postconditions (`ensure`) . . . 30
Combining `require` and `ensure` . . . 31
Struct Invariants (`invariant`) . . . 31
Rules . . . 31

Testing **32**
Running Tests . . . 32
Writing Tests . . . 32
Matchers . . . 32
Output Format . . . 33
Mocking . . . 33
Parameterized Tests . . . 33
Property-Based Tests . . . 34
Limitations . . . 34

English | |

01 - Getting Started

Next tutorial -> 02 - Variables and Types

Prerequisites

To build and run Ry, you need the following:

- **LLVM 21**
- **CMake 3.20 or later**
- **A C++17 compatible compiler** (GCC 7+ / Clang 5+, etc.)

Build Instructions

Run the following commands from the repository root:

```
cmake -B build -DLLVM_DIR=/usr/local/llvm/lib/cmake/llvm
cmake --build build
```

On a successful build, an executable named `build/ry` will be generated.

Project Initialization

You can create a new project with the `ry init` command:

```
mkdir my-project
cd my-project
ry init
```

This generates the following files and directories:

- `ry.toml` – Project configuration file
- `src/main.ry` – Entry point (with sample code)
- `test/` – Directory for test code

See Project Management for details.

Your First Program

Save the following content to a file named `hello.ry`:

```
print("Hello, World!")
```

Run it with the following command:

```
./build/ry hello.ry
```

Output:

```
Hello, World!
```

Writing Comments

Everything from `#` to the end of the line is treated as a comment.

```
# This is a comment
print("Hello")  # End-of-line comments are also supported
```

Comments do not affect the behavior of the code.

Next tutorial -> 02 - Variables and Types

English | |

02 - Variables and Types

<- 01 - Getting Started / Next -> 03 - Operators

Constant Declaration with let

The `let` keyword declares an immutable variable (constant). The type is automatically inferred from the value on the right-hand side. The value cannot be changed after declaration.

```
let x = 42          # Inferred as int
let y = 3.14        # Inferred as float
let flag = true     # Inferred as bool
let name = "Ry"     # Inferred as str
```

Variable Declaration with var

The `var` keyword declares a mutable variable. You can reassign a value of the same type after declaration.

```
var count = 0
count = count + 1    # OK: reassignment to the same type
```

Type Annotations

You can explicitly specify the type of a variable.

```
let x: int = 42
let rate: float = 0.5
let ok: bool = false
let msg: str = "hello"
```

A compile error occurs if the type annotation does not match the actual type of the value.

Basic Types

Type	Description	Literal Examples
`int`	64-bit integer	`0`, `42`, `-10`
`byte`	Unsigned 8-bit integer (0-255)	`let b: byte = 42`
`float`	64-bit floating-point number	`0.0`, `3.14`, `-1.5`
`bool`	Boolean	`true`, `false`
`str`	String	`"hello"`, `""`

String Operations

Various operations are available for strings.

```
let a = "Hello"
let b = "World"

# Concatenation
let c = a + ", " + b   # "Hello, World"

# Comparison (lexicographic order)
print(a == b)    # false
print(a != b)    # true
print(a < b)     # true ("H" < "W")

# Length
print(len(a))    # 5

# Substring checks
let s = "Hello, World!"
print(contains(s, "World"))       # true
print(starts_with(s, "Hello"))    # true
print(ends_with(s, "!"))          # true
```

Escape Sequences

The following escape sequences can be used within strings.

Sequence	Meaning
`\n`	Newline
`\t`	Tab
`\\`	Backslash
`\"`	Double quote
`\0`	Null character

```
print("Hello\nWorld")    # Outputs on two lines
print("A\tB")            # Tab-separated
print("say \"hi\"")      # String containing double quotes
```

Reassignment Rules

Variables declared with `var` can be reassigned. However, the following restrictions apply:

```
var x = 10
x = 20        # OK: reassignment to the same type
# x = "text" # Error: reassignment with a different type is not allowed
```

`let` variables cannot be reassigned.

```
let N = 5
# N = 6  # Error: reassignment to a let variable is not allowed
```

Redeclaring a variable with the same name is also not allowed.

```
let x = 1
# let x = 2  # Error: redeclaring the same name is not allowed
```

Tuple Destructuring

You can unpack a tuple into multiple variables in a single declaration using `let` or `var`.

```
let a, b = (10, 20)
print(a)    # 10
print(b)    # 20
```

Wildcard

Use _ to ignore a position.

```
let x, _ = (1, 2)    # only x is bound; 2 is discarded
print(x)              # 1
```

Mutable Destructuring

Use `var` to declare mutable variables.

```
var a, b = (10, 20)
a = 99
print(a)    # 99
```

Rules

- The number of variables on the left must match the number of elements in the tuple.
- Each variable follows the same `let`/`var` rules as a regular declaration.
- Nested tuple destructuring is not supported.

<- 01 - Getting Started / Next -> 03 - Operators

English | |

03 - Operators

<- 02 - Variables and Types / Next -> 04 - Control Flow

Arithmetic Operators

Operator	Description	Example	Result
`+`	Addition	`3 + 2`	`5`
`-`	Subtraction	`3 - 2`	`1`
`*`	Multiplication / string repetition	`3 * 2`	`6`
`/`	Division (always float)	`7 / 2`	`3.5`
`//`	Integer division (always int)	`7 // 2`	`3`
`%`	Modulo	`7 % 3`	`1`
`**`	Exponentiation (always float)	`2 ** 10`	`1024.0`

```
let a = 10
let b = 3

print(a + b)     # 13
print(a - b)     # 7
print(a * b)     # 30
print(a / b)     # 3.3333... (float)
print(a // b)    # 3 (int)
print(a % b)     # 1
print(2 ** 8)    # 256.0 (float)
```

Comparison Operators

All comparison operators return a `bool` value.

Operator	Description	Example
`==`	Equal to	`a == b`
`!=`	Not equal to	`a != b`
`<`	Less than	`a < b`
`<=`	Less than or equal to	`a <= b`
`>`	Greater than	`a > b`
`>=`	Greater than or equal to	`a >= b`

```
let x = 5
let y = 10

print(x == y)    # false
print(x != y)    # true
print(x < y)     # true
print(x <= y)    # true
print(x > y)     # false
print(x >= y)    # false
```

Comparison operators also work on strings (lexicographic comparison).

```
print("abc" == "abc")    # true
print("abc" < "abd")     # true
print("b" > "a")         # true
```

Logical Operators

Operator	Description	Example
`and`	Logical AND	`a and b`
`or`	Logical OR	`a or b`
`not`	Logical NOT	`not a`

```
let t = true
let f = false

print(t and f)    # false
```

```
print(t or f)     # true
print(not t)      # false
print(not f)      # true
```

Bitwise Operators

Bitwise operators can only be used with the `int` type.

Operator	Description	Example
`&`	Bitwise AND	`5 & 3 -> 1`
`|`	Bitwise OR	`5 | 3 -> 7`
`^`	Bitwise XOR	`5 ^ 3 -> 6`
`~`	Bitwise NOT (unary)	`~5 -> -6`
`<<`	Left shift	`1 << 3 -> 8`
`>>`	Right shift (arithmetic)	`8 >> 2 -> 2`
`>>>`	Logical right shift	`-1 >>> 1 -> 9223372036854775807`

```
let a = 0b1010    # 10
let b = 0b1100    # 12

print(a & b)     # 8  (0b1000)
print(a | b)     # 14 (0b1110)
print(a ^ b)     # 6  (0b0110)
print(~a)        # -11
print(1 << 4)    # 16
print(32 >> 2)   # 8
```

Compound Assignment Operators

Shorthand notation for updating the value of a variable.

Operator	Description	Equivalent Expression
`+=`	Addition assignment	`x = x + n`
`-=`	Subtraction assignment	`x = x - n`
`*=`	Multiplication assignment	`x = x * n`
`/=`	Division assignment	`x = x / n`
`%=`	Modulo assignment	`x = x % n`

```
let x = 10
x += 5     # x == 15
x -= 3     # x == 12
x *= 2     # x == 24
x /= 4     # x == 6.0 (becomes float)
```

Increment / Decrement Operators

Shorthand for incrementing or decrementing a variable by 1.

Operator	Description	Equivalent Expression
`x++`	Increment by 1	`x = x + 1`
`x--`	Decrement by 1	`x = x - 1`

```
var count = 0
count++       # count == 1
count++       # count == 2
count--       # count == 1
```

Note: These are statement-only operators. They cannot be used inside expressions.

Type Promotion Rules

The following describes the behavior when `int` and `float` are mixed in operations.

```
# + - * produce float if either operand is float
print(1 + 2)      # 3 (int)
print(1 + 2.0)    # 3.0 (float)
print(1.0 + 2)    # 3.0 (float)

# / always produces float
print(4 / 2)      # 2.0 (float)

# // always produces int
print(7 // 2)     # 3 (int)
print(7.0 // 2)   # 3 (int)

# ** always produces float
print(2 ** 3)     # 8.0 (float)

# % produces int if both operands are int, float if either is float
print(7 % 3)      # 1 (int)
print(7.5 % 2)    # 1.5 (float)

# + concatenates strings if both operands are str
print("foo" + "bar")    # "foobar"

# * repeats a string if one operand is str and the other is int
print("ab" * 3)         # "ababab"
print(3 * "ab")         # "ababab"
```

Membership Operators

Operator	Description	Example
`in`	Membership test	`2 in {1, 2, 3}` -> `true`
`not in`	Negated membership test	`4 not in {1, 2, 3}` -> `true`

```
let s = {1, 2, 3}
print(2 in s)        # true
print(4 not in s)    # true
```

<- 02 - Variables and Types / Next -> 04 - Control Flow

English | |

Control Flow

<- Prev: Operators | Next: Functions ->

---

if / elif / else

Use `if` to branch execution based on conditions.

```
let x = 10

if x > 0:
    print(x)
elif x == 0:
    print(0)
else:
    print(-1)
```

- `elif` and `else` are optional.
- Conditions are not limited to `bool` values. For `int`, `0` is treated as false and non-`0` as true.
- `if` statements can be nested.

```
let a = 5
let b = 3

if a > 0:
    if b > 0:
        print(a + b)    # 8
```

---

while Loop

Repeatedly executes a block as long as the condition is true.

```
let i = 3
while i > 0:
    print(i)
    i = i - 1
# 3
# 2
# 1
```

---

for Loop and range

You can iterate over a list or using `range`.

```
for x in [1, 2, 3]:
    print(x)
# 1
```

```
# 2
# 3
```

`range(n)` generates integers from `0` to `n - 1`.

```
for i in range(5):
    print(i)
# 0
# 1
# 2
# 3
# 4
```

`range(start, end)` generates integers from `start` to `end - 1`.

```
for i in range(2, 5):
    print(i)
# 2
# 3
# 4
```

The `..` range operator creates an inclusive range: `1 .. 3` produces `[1, 2, 3]`.

```
for i in 1 .. 3:
    print(i)
# 1
# 2
# 3
```

You can iterate over map key-value pairs with `for k, v in map`:

```
let m = {"x": 10, "y": 20}
for k, v in m:
    print(k)
    print(v)
```

---

break and continue

`break` immediately exits the loop. `continue` skips the current iteration and proceeds to the next one.

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
# 1
# 3
```

They can also be used with `while` loops.

```
let n = 0
while true:
    n = n + 1
    if n % 2 == 0:
        continue
    if n > 7:
        break
```

```
    print(n)
# 1
# 3
# 5
# 7
```

Note: In nested loops, `break` / `continue` only affect the innermost loop. Using them outside a loop results in a compile error.

Nesting Example

`for` and `while` loops can be nested.

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i * 10 + j)
# 11
# 13
# 21
# 23
# 31
# 33
```

Scope Rules

Control flow blocks have their own scope.

Block Scope

Variables declared inside a block cannot be referenced from outside the block.

```
if true:
    let inner = 42
# Referencing inner here causes a compile error
```

Referencing and Reassigning Outer Variables

You can reference and reassign outer variables from within a block.

```
let count = 0
for i in range(5):
    count = count + i
print(count)   # 10
```

Shadowing

If you declare a variable with the same name as an outer variable inside a block, the new variable is used within that block (shadowing). The outer variable remains unchanged.

```
let x = 1
if true:
    let x = 99
```

```
    print(x)    # 99
print(x)        # 1
```

---

match

`match` is a construct for branching based on a value. It can safely handle enums and Options.

```
enum Color:
    Red
    Green
    Blue

let c = Color::Green
match c:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")
# green
```

Option Matching

Use `match` to safely handle both the `Some` and `None` cases.

```
let x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")
# 42
```

Wildcards and Literals

`_` is a wildcard pattern that matches anything. You can also match against literal values (numbers, strings, booleans).

```
let n = 5
match n:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")
# other
```

Guard Clauses

You can add guard conditions with `if`.

```
match n:
    case x if x > 0:
        print("positive")
```

```
    case x if x < 0:
        print("negative")
    case _:
        print("zero")
```

> **Note**: `match` must be exhaustive. For enums, all variants must be covered. For Options, both `Some` and `None` are required. For literals, a `_` wildcard is needed.

<- Prev: Operators | Next: Functions ->

English | |

Functions

<- Prev: Control Flow | Next: Structs ->

Basic Function Definition

Functions are defined with the `fn` keyword. Parameter type declarations are required and use the `name: type` format. The return type is specified after `->`.

```
fn add(a: int, b: int) -> int:
    return a + b
```

- Parameter type declarations are required.
- The return type is specified after `->`.
- Use `return` to return a value.

Calling Functions

Call a defined function by its name with arguments.

```
fn multiply(x: int, y: int) -> int:
    return x * y

let result = multiply(3, 4)
print(result)    # 12
```

Recursive Functions

Functions can call themselves (recursion).

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))    # 120
print(factorial(0))    # 1
```

Function Overloading

You can define multiple functions with the same name but different parameter counts or types.

```
fn add(a: int, b: int) -> int:
    return a + b

fn add(a: float, b: float) -> float:
    return a + b

print(add(1, 2))        # 3
print(add(1.5, 2.5))    # 4
```

The appropriate function is automatically selected based on the argument types at the call site.

> **Note**: Defining functions with identical parameter types but different return types causes a compile error.

Omitting the Return Type (Unit Type)

If a function does not need to return a value, you can omit `->`. In this case, the function returns the Unit type.

```
fn greet():
    print(42)

greet()    # 42
```

This is the simplest form of a function with no parameters and no return value.

<- Prev: Control Flow | Next: Structs ->

English | |

Structs and Enums

<- Prev: Functions | Next: Collections ->

Defining Structs with record

Structs are defined with the `record` keyword. Each field is described in the `name: type` format.

```
record Point:
    x: int
    y: int
```

Structs are value types allocated on the stack.

Using Constructors

Create an instance by calling the struct name as a function. Arguments are specified in the order the fields are defined.

```
let p = Point(10, 20)
```

Field Access (Dot Notation)

Fields are accessed using dot notation.

```
let p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

> **Note**: Passing a struct directly to `print` causes an error. Pass individual fields instead.

Field Assignment

Fields of variables declared with `var` can be reassigned.

```
var p = Point(10, 20)
p.x = 100
print(p.x)    # 100
```

> **Note**: Assigning to fields of a `let` variable causes a compile error.

Structs as Function Parameters

Structs can be passed as function arguments.

```
record Point:
    x: int
    y: int

fn distance_x(a: Point, b: Point) -> int:
    return a.x - b.x

let p1 = Point(10, 3)
let p2 = Point(4, 7)
print(distance_x(p1, p2))    # 6
```

Nested Structs

A struct's field can be another struct.

```
record Point:
    x: int
    y: int

record Line:
    start: Point
    end: Point

let line = Line(Point(0, 0), Point(10, 5))
print(line.start.x)    # 0
print(line.end.x)      # 10
```

You can access nested fields by chaining dot notation.

Enums

Enums are defined with the `enum` keyword. Each variant is treated as a named constant.

Definition

```
enum Color:
    Red
    Green
    Blue
```

Usage

Variants are accessed with `::`.

```
let c = Color::Red
print(c)    # Red
```

Comparison

Variants can be compared using `==` and `!=`.

```
if c == Color::Red:
    print("red!")
elif c == Color::Green:
    print("green!")
else:
    print("blue!")
```

Function Parameters

Enum names can be used as function parameter types.

```
fn describe(c: Color) -> str:
    if c == Color::Red:
        return "warm"
    return "cool"
```

<- Prev: Functions | Next: Collections ->

English | |

Collections

<- Prev: Structs | Next: Advanced Features ->

Ry has four collection types: **Tuples**, **Lists**, **Maps**, and **Sets**.

Tuples

A tuple is an immutable data structure that groups multiple values together. It can hold elements of different types.

Creation

```
let t = (1, 3.14)
```

Type Annotation

```
let t: (int, float) = (1, 3.14)
```

Element Access

Elements are accessed by index using `.0`, `.1`, etc.

```
let t = (1, 3.14)
print(t.0)    # 1
print(t.1)    # 3.14
```

As Function Return Values

Tuples are useful when you want to return multiple values.

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)

let result = swap(1, 2)
print(result.0)   # 2
print(result.1)   # 1
```

Limitations

- Accessing an out-of-bounds index (e.g., `.2` on a tuple with 2 elements) causes a compile error.
- Passing a tuple directly to `print` causes an error. Pass each element individually.

Lists

A list is a variable-length data structure containing elements of the same type.

Creation

```
let xs = [1, 2, 3]
```

Type Annotation

```
let xs: List<int> = [1, 2, 3]
```

Index Access

```
print(xs[0])    # 1

let i = 1
print(xs[i])    # 2
```

Index Assignment

```
xs[0] = 99
```

len

```
print(len(xs))   # 3
```

print

```
print(xs)   # [1, 2, 3]
```

Iteration with for

```
for x in xs:
    print(x)
```

Function Parameters

```
fn first(xs: List<int>) -> int:
    return xs[0]
```

filter, map, sort

Lists support `filter`, `map`, and `sort` operations. These return new lists without modifying the original.

```
let xs = [1, 2, 3, 4, 5]

# filter: keep elements matching a condition
let evens = xs.filter(fn(x: int): x > 3)
print(evens)   # [4, 5]

# map: transform each element
let doubled = xs.map(fn(x: int): x * 2)
print(doubled)   # [2, 4, 6, 8, 10]

# sort: sort in ascending order (default)
let sorted = [3, 1, 2].sort()
print(sorted)   # [1, 2, 3]

# Chaining
let result = xs.filter(fn(x: int): x > 1).map(fn(x: int): x * 10).sort()
print(result)   # [20, 30, 40, 50]
```

reduce, fold

`reduce` accumulates a list into a single value starting from the first element. `fold` does the same but with an explicit initial value.

```
let xs = [1, 2, 3, 4, 5]

# reduce: start from first element
let total = reduce(xs, fn(a: int, b: int): a + b)
print(total)   # 15

# fold: provide an explicit initial value
let total2 = fold(xs, 0, fn(a: int, b: int): a + b)
print(total2)   # 15
```

any, all

`any` returns `true` if at least one element satisfies the predicate. `all` returns `true` if every element does.

```
let xs = [1, 2, 3, 4, 5]

print(any(xs, fn(x: int): x > 4))   # true
print(any(xs, fn(x: int): x > 9))   # false

print(all(xs, fn(x: int): x > 0))   # true
print(all(xs, fn(x: int): x > 3))   # false
```

sum, min, max

```
let xs = [3, 1, 4, 1, 5]
print(sum(xs))   # 14
print(min(xs))   # 1
print(max(xs))   # 5
```

first, last, is_empty

```
let xs = [10, 20, 30]
print(first(xs))      # 10
print(last(xs))       # 30
print(is_empty(xs))   # false
```

enumerate, zip

`enumerate` pairs each element with its index. `zip` combines two lists element-by-element.

```
let xs = [10, 20, 30]
let indexed = enumerate(xs)
# [(0, 10), (1, 20), (2, 30)]
for p in indexed:
    print(p.0)
    print(p.1)

let ys = ["a", "b", "c"]
let zipped = zip(xs, ys)
# [(10, "a"), (20, "b"), (30, "c")]
```

Limitations

- All elements must be of the same type. Mixing different types is not allowed.
- An empty list `[]` causes an error.
- Out-of-bounds access results in a runtime error (`exit(1)`).
- Supported element types are `int`, `float`, `bool`, and `str`.

Maps

A map is an associative array that manages key-value pairs.

Creation

```
let m = {"a": 1, "b": 2}
```

Type Annotation

```
let m: Map<str, int> = {"a": 1, "b": 2}
```

Key Access

```
print(m["a"])   # 1
```

Insertion / Update

Assigning to a new key inserts it; assigning to an existing key updates it.

```
m["c"] = 3    # Insert new entry
m["a"] = 99   # Update existing entry
```

len

```
print(len(m))   # 3
```

print

```
print(m)   # {a: 99, b: 2, c: 3}
```

has_key

Checks whether a key exists.

```
print(m.has_key("a"))   # true
```

keys, values

`keys` returns a list of all keys. `values` returns a list of all values.

```
let m = {"a": 1, "b": 2, "c": 3}
print(keys(m))     # ["a", "b", "c"]
print(values(m))   # [1, 2, 3]
```

Function Parameters

```
fn get_val(m: Map<str, int>, k: str) -> int:
    return m[k]
```

Limitations

- All keys must be of the same type, and all values must be of the same type.
- An empty map requires a type annotation.
- Accessing a nonexistent key results in a runtime error (`exit(1)`).

Sets

A set is a collection that holds elements of the same type without duplicates.

Creation

```
let s = {1, 2, 3}
```

Type Annotation

```
let s: Set<int> = {1, 2, 3}
```

in Operator

Use the `in` operator to check if an element is in the set.

```
print(2 in s)   # true
print(5 in s)   # false
```

add / remove

```
s.add(4)        # Add element
s.remove(1)     # Remove element
s.add(2)        # Ignored since it already exists
```

len / print

```
print(len(s))  # 3
print(s)       # {2, 3, 4}
```

Iteration with for

```
for x in s:
    print(x)
```

Empty Set

An empty set requires a type annotation.

```
let empty: Set<int> = {}
```

Limitations

- All elements must be of the same type.
- Supported element types are `int`, `float`, `bool`, and `str`.

Iterators

Iterators provide a **lazy** way to process collections. Instead of creating intermediate lists at each step, iterators process elements one at a time through a pipeline.

Creating and Consuming

```
let xs = [1, 2, 3]
let ys = xs.iter().to_list()   # [1, 2, 3]
```

Chaining Operations

You can chain `filter`, `map`, and `take` to build pipelines:

```
let result = [1, 2, 3, 4, 5]
    .iter()
    .filter(fn(x: int): x > 2)
    .map(fn(x: int): x * 2)
    .take(2)
```

```
    .to_list()
print(result)   # [6, 8]
```

Manual Iteration with next()

```
let it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

For Loops

Iterators work directly in `for` loops:

```
for x in [1, 2, 3].iter().filter(fn(x: int): x > 1):
    print(x)    # 2, 3
```

Maps produce tuple elements:

```
for k, v in {"a": 1, "b": 2}.iter():
    print(k)
```

<- Prev: Structs | Next: Advanced Features ->

English | |

Advanced Features

<- Prev: Collections | Next: Packages ->

Lambda Functions

Lambda functions are a syntax for writing functions as expressions. They use the form `fn(parameters): expression`. The return type is automatically inferred.

Single-Expression Lambda

```
let double = fn(x: int): x * 2
print(double(5))   # 10

let add = fn(a: int, b: int): a + b
print(add(3, 4))   # 7
```

No-Parameter Lambda

```
let answer = fn(): 42
print(answer())   # 42
```

Multi-Line Lambda

You can write multiple statements by adding a newline after `:` and indenting.

```
let abs = fn(x: int):
    if x < 0:
        return -x
```

```
    return x

print(abs(-5))  # 5
print(abs(3))   # 3
```

Closures

Lambda functions can capture variables from the scope in which they are defined.

```
let offset = 10
let add_offset = fn(x: int): x + offset
print(add_offset(5))  # 15
```

Higher-Order Functions

You can define functions that take other functions as arguments. Function types are written as `fn(parameter_types) -> return_type`.

```
fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

let double = fn(x: int): x * 2
print(apply(double, 3))                # 6
print(apply(fn(n: int): n + 1, 10))    # 11
```

Functions as Values

Named functions can also be bound to variables or passed as arguments.

```
fn square(x: int) -> int:
    return x * x

fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

# Pass a named function as an argument
print(apply(square, 4))  # 16

# Bind to a variable
let sq = square
print(sq(5))  # 25
```

UFCS (Uniform Function Call Syntax)

With UFCS, you can write `f(a, b)` as `a.f(b)`. This enables method-chaining-style notation.

```
fn add(a: int, b: int) -> int:
    return a + b

let x = 1
print(x.add(2))   # add(x, 2) -> 3
```

Chained Calls

```
fn double(n: int) -> int:
    return n * 2

print(x.add(2).double())    # double(add(x, 2)) -> 6
```

Operator Overloading

You can define operators for custom types using the `fn operator` syntax.

Binary Operators

Takes two parameters.

```
record Vec2:
    x: int
    y: int

fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

let v1 = Vec2(1, 2)
let v2 = Vec2(3, 4)
let v3 = v1 + v2
print(v3.x)         # 4
print(v1 == v2)     # false
```

Unary Operators

Takes one parameter.

```
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

Supported Operators

Category	Operators
Arithmetic	`+`, `-`, `*`, `/`, `%`, `**`, `//`
Comparison	`==`, `!=`, `<`, `<=`, `>`, `>=`
Bitwise	`&`, `|`, `^`, `~`, `<<`, `>>`
Logical	`and`, `or`, `not`

Option Type

A type that represents whether a value exists or not. It takes either `Some(value)` or `None`.

```
let x: Option<int> = Some(42)
print(x)    # Some(42)
```

```
let y: Option<int> = None
print(y)   # None
```

Extracting the Value

Use `match` to safely extract the inner value and handle the `None` case.

```
match x:
    case Some(v):
        print(v)    # 42
    case None:
        print("nothing")
```

Concurrency Basics

`Task<T>` is the runtime handle for concurrent work. Use `spawn` for explicit task creation, `async fn` for task-returning functions, and `await` or `join(task)` to wait for completion.

```
fn square(x: int) -> int:
    return x * x

let t: Task<int> = spawn square(12)
print(await t)   # 144

async fn add(a: int, b: int) -> int:
    return a + b

print(await add(20, 22))   # 42
await add(1, 2)            # statement form also works
```

`@parallel` can be applied to counted `for` loops over `range(...)` or integer `..` ranges:

```
@parallel
for i in range(8):
    print(i)
```

In v1, `spawn` does not support `Unit`-returning calls, and `@parallel for` rejects `break`, `continue`, and writes to outer mutable variables.

For channels, `recv(ch)` is the strict form and raises on a closed drained channel, while `recv_opt(ch)` returns `Some(value)` or `None` instead. `for x in ch:` is the close-aware consumer form and ends normally once the channel is closed and drained. For `Channel<Unit>`, `recv_opt(ch)` returns `bool` and `for _ in ch:` can be used to consume values.

F-String (String Interpolation)

Use `f"..."` to embed expressions directly inside strings. Expressions are placed in `{}`.

```
let name = "Alice"
print(f"Hello {name}")   # Hello Alice

let x = 3
let y = 4
print(f"{x} + {y} = {x + y}")   # 3 + 4 = 7
```

Use `{{` and `}}` to include literal braces.

```
print(f"{{escaped}}")   # {escaped}
```

Type Casting (as)

Convert between types explicitly with `as`.

```
let x = 42 as float      # 42.0
let y = 3.14 as int       # 3 (truncated)
let s = 42 as str          # "42"
let b = true as int        # 1
```

Enum with Associated Data (ADT)

Enum variants can carry associated values. This lets a single enum represent a family of different shapes of data.

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

Constructing ADT Variants

```
let c = Shape::Circle(3.14)
let r = Shape::Rectangle(4.0, 5.0)
let p = Shape::Point
```

Matching ADT Variants

Use `case` with a binding pattern to extract the associated data.

```
fn describe(s: Shape) -> str:
    match s:
        case Shape::Circle(r):
            return f"circle with radius {r}"
        case Shape::Rectangle(w, h):
            return f"rectangle {w}x{h}"
        case Shape::Point:
            return "point"

print(describe(Shape::Circle(3.14)))         # circle with radius 3.14
print(describe(Shape::Rectangle(4.0, 5.0)))  # rectangle 4.0x5.0
```

Generic Enum

Enums can take type parameters, making them reusable across different payload types.

```
enum MyOption<T>:
    MySome(T)
    MyNone
```

Usage

```
let a = MyOption<int>::MySome(42)
let b: MyOption<int> = MyOption<int>::MyNone

match a:
    case MyOption::MySome(v):
        print(v)      # 42
    case MyOption::MyNone:
        print("none")
```

Result Type

`Result<T, E>` is used for functions that may fail. Return `Ok(value)` for success and `Err(error)` for failure.

```
fn divide(a: int, b: int) -> Result<int, str>:
    if b == 0:
        return Err("division by zero")
    return Ok(a // b)
```

Use `match` to handle the result.

```
let r = divide(10, 0)
match r:
    case Ok(v):
        print(v)
    case Err(e):
        print(e)    # division by zero
```

<- Prev: Collections | Next: Packages ->

English | |

Packages

<- Prev: Advanced Features | Next: Design by Contract ->

Ry uses a package system to organize code across files and directories. For the full specification, see Package Reference.

---

from/import Syntax

Use the `from` syntax to import functions from another file.

```
from math import add, sub    # Selective import
from math                     # Full import (all definitions)
```

This makes functions defined in `math.ry` available for use.

Subdirectories (Dot-Separated Paths)

Use dot-separated paths to specify packages in subdirectories.

```
from utils.math import add    # Import from utils/math.ry
```

Each dot corresponds to a directory separator.

Directory Packages

A package can be either a single `.ry` file or a directory containing multiple `.ry` files. When a package resolves to a directory, all `.ry` files in it are automatically loaded.

```
mypackage/
  math.ry      # fn add(), fn sub()
  string.ry    # fn concat()
```

```
from mypackage              # imports add, sub, concat
from mypackage import add   # imports only add
```

No special entry file (like `__init__.py`) is needed. Files starting with `_` are excluded.

Standard Library (`std`)

The `std` package is automatically imported into every program. You don't need to write `from std` — it's always available.

```
# These functions are available without any import
print("hello")
let n = len("world")
let xs = range(5)
```

You can also explicitly import specific definitions from `std` sub-packages:

```
from std.str import contains
```

RY_HOME

The standard library is installed at `$RY_HOME/lib/std/`. The default value of `RY_HOME` is `~/.ry`.

```
export RY_HOME="$HOME/.ry"    # default
```

Search Path Priority

Package files are searched in the following order:

1. **Directory of the importing file** — The directory containing the file with the import statement is searched first.
2. **`$RY_HOME/lib`** — Standard library location.
3. **Executable-relative `lib/`** — Directories relative to the `ry` executable.
4. **`RY_PATH` environment variable** — If not found, directories specified in `RY_PATH` are searched in order.

RY_PATH Environment Variable

Multiple directories can be specified separated by colons.

```
export RY_PATH=/home/user/ry-libs:/usr/local/ry-libs
```

Once set, packages in the specified directories can be imported from anywhere.

Limitations

- `from` statements can only be written at the **top level** of a file. They cannot be placed inside functions or blocks.
- Importing the same package multiple times is automatically skipped (no duplicate imports).
- **Circular imports** (A imports B and B imports A) result in an error.

```
# Error example: a.ry and b.ry import each other
# a.ry: from b import foo
# b.ry: from a import bar  <- circular import error
```

<- Prev: Advanced Features | Next: Design by Contract ->

English | |

Design by Contract

<- Prev: Packages | Next: Testing ->

Ry supports Eiffel-style Design by Contract with preconditions (`require`), postconditions (`ensure`), and struct invariants (`invariant`). Contract violations terminate the program. For the full specification, see Design by Contract Reference.

Preconditions (`require`)

Use `require` to specify conditions that must be true when a function is called.

```
fn deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    var new_balance: int = balance + amount
    return new_balance
```

If any precondition fails, the program terminates with:

```
Contract violation: require failed in deposit()
```

Postconditions (`ensure`)

Use `ensure` to specify conditions that must be true when a function returns.

The `result` Keyword

Inside an `ensure` block, `result` refers to the return value.

```
fn abs(x: int) -> int:
    ensure:
        result >= 0
    if x < 0:
```

```
        return -x
    return x
```

The `old()` Expression

`old(expr)` captures the value of an expression at function entry. This is useful for comparing pre- and post-states.

```
fn increment(x: int) -> int:
    ensure:
        result == old(x) + 1
    return x + 1
```

Combining `require` and `ensure`

Both can be used together. `require` must come before `ensure`.

```
fn deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    ensure:
        result >= 0
        result == old(balance) + amount
    var new_balance: int = balance + amount
    return new_balance
```

Struct Invariants (`invariant`)

Use `invariant` to specify conditions that must always hold for a struct. Invariants are checked after construction and after every field assignment.

```
record BankAccount:
    balance: int
    min_balance: int
    invariant:
        balance >= min_balance

let a = BankAccount(100, 0)    # OK: 100 >= 0
# a.balance = -1               # Contract violation: invariant failed
```

Rules

- `require` and `ensure` blocks are optional and appear before the function body.
- `require` must come before `ensure` when both are present.
- `result` and `old()` can only be used inside `ensure` blocks.
- `invariant` appears at the end of a `record` definition, after all field declarations.
- All contract violations terminate with `exit(1)`.

<- Prev: Packages | Next: Testing ->

English | |

Testing

<- Prev: Design by Contract

Ry has a built-in RSpec-style test syntax using `describe`, `it`, and `expect`. For the full specification, see Testing Reference.

Running Tests

```
ry test                          # Auto-discover and run all *.test.ry files
ry test tests/my_test.test.ry # Run a specific test file
```

The exit code is `0` if all tests pass, `1` if any test fails.

When run without arguments, `ry test` searches for `ry.toml` to find the project root, then recursively discovers all `*.test.ry` files.

Writing Tests

Use `describe` to group related tests and `it` to define individual test cases.

```
describe("Calculator", fn():
    it("adds integers", fn():
        expect(1 + 2).to_eq(3)

    )
    it("subtracts integers", fn():
        expect(5 - 3).to_eq(2)

    )
    it("checks booleans", fn():
        expect(3 > 1).to_be_true()
    )
)
```

- `describe` and `it` take a description string and a **lambda argument** `fn():` as the second parameter
- `describe`, `it`, `expect`, `mock`, and `verify` are only available with `ry test` (compile error with normal `ry` execution)

Matchers

Matcher	Description	Supported Types
`to_eq(expected)`	Equality comparison	int, float, bool, str
`to_not_eq(expected)`	Asserts not equal	int, float, bool, str
`to_be_true()`	Asserts `true`	bool
`to_be_false()`	Asserts `false`	bool
`to_be_none()`	Asserts `None`	Option
`to_be_some()`	Asserts Option is `Some`	Option
`to_contain(val)`	Asserts container includes value	List, Set, str

Output Format

```
Calculator
  + adds integers
  + subtracts integers
  - checks booleans
    line 10: expected true, got false

2 passed, 1 failed
```

`+` indicates pass (green), `-` indicates failure (red).

Mocking

`mock(fn_name, replacement)`

Replaces a function with a mock implementation for the current `it` block. The mock is automatically restored when the `it` block ends.

```
fn fetch_data() -> str:
    return "real data"

describe("mocking", fn():
    it("replaces function", fn():
        mock(fetch_data, fn(): "fake")
        expect(fetch_data()).to_eq("fake")

    )
    it("auto-restores", fn():
        expect(fetch_data()).to_eq("real data")
    )
)
```

`verify(fn_name)`

Returns the number of times a mocked function was called.

```
describe("verify", fn():
    it("counts calls", fn():
        mock(fetch_data, fn(): "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
    )
)
```

Parameterized Tests

Use `@each` to run the same test with multiple inputs:

```
@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)
```

Each tuple becomes a separate test case. `{0}`, `{1}`, etc. in the description are replaced with actual values.

Property-Based Tests

Use `@property` to test with randomly generated inputs:

```
@property(count=100)
it("addition is commutative", fn(a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

The test runs `count` times with random values. On failure, the counterexample is printed.

Limitations

- Nesting of `describe` is not supported
- `before_each` / `after_each` are not supported
- Overloaded and `@native` functions cannot be mocked

<- Prev: Design by Contract